NED University of Engineering & Technology
Karachi
Department of Software Engineering

COMPLEX ENGINEERING ACTIVITY (CEA)

Technical Report

Design, Deployment, and Cloud-Native Enhancement of a Three-Tier Web Application

Project: Eventra – Event Management System on AWS

Course	Cloud Computing
Course Code	SE-490
Section	B
Instructor	Miss Amna

Group Members

Name	Roll No.
Batool Fatima	SE-23076
Syed Muhammad Aun Zaidi	SE-23095

Source code: https://github.com/AunZaidii/Eventra-Event-Management-System

1. Chosen Cloud Provider and Justification

Eventra is an event management web application. Visitors can browse upcoming events and register for them (the system rejects duplicate emails and sold-out events), and an administrator can create and manage events, view a statistics dashboard and export the registrations as a CSV file. For this activity we containerized the application, deployed it on a public cloud with a private network layout, and set up a CI/CD pipeline that builds, tests and deploys it on every change.

We chose **Amazon Web Services (AWS)** for the following reasons:

- AWS provides every component the brief asks for as a managed service: VPC, Application Load Balancer, ECS Fargate, RDS PostgreSQL, ECR, Secrets Manager and CloudWatch.
- ECS Fargate runs containers without us having to manage servers, which suited a team of two.
- GitHub Actions can log in to AWS using OpenID Connect, so no long-lived access keys need to be stored in the pipeline.
- AWS has plenty of documentation and community help, which made troubleshooting and later maintenance easier.

We deployed in the us-east-1 region because it has some of the lowest prices on AWS and supports every service we used.

2. Overall Architecture and Design Decisions

Figure 1 shows the deployed architecture. All infrastructure is written in Terraform (about 60 resources in infra/terraform), so the whole environment can be created or destroyed with one command.

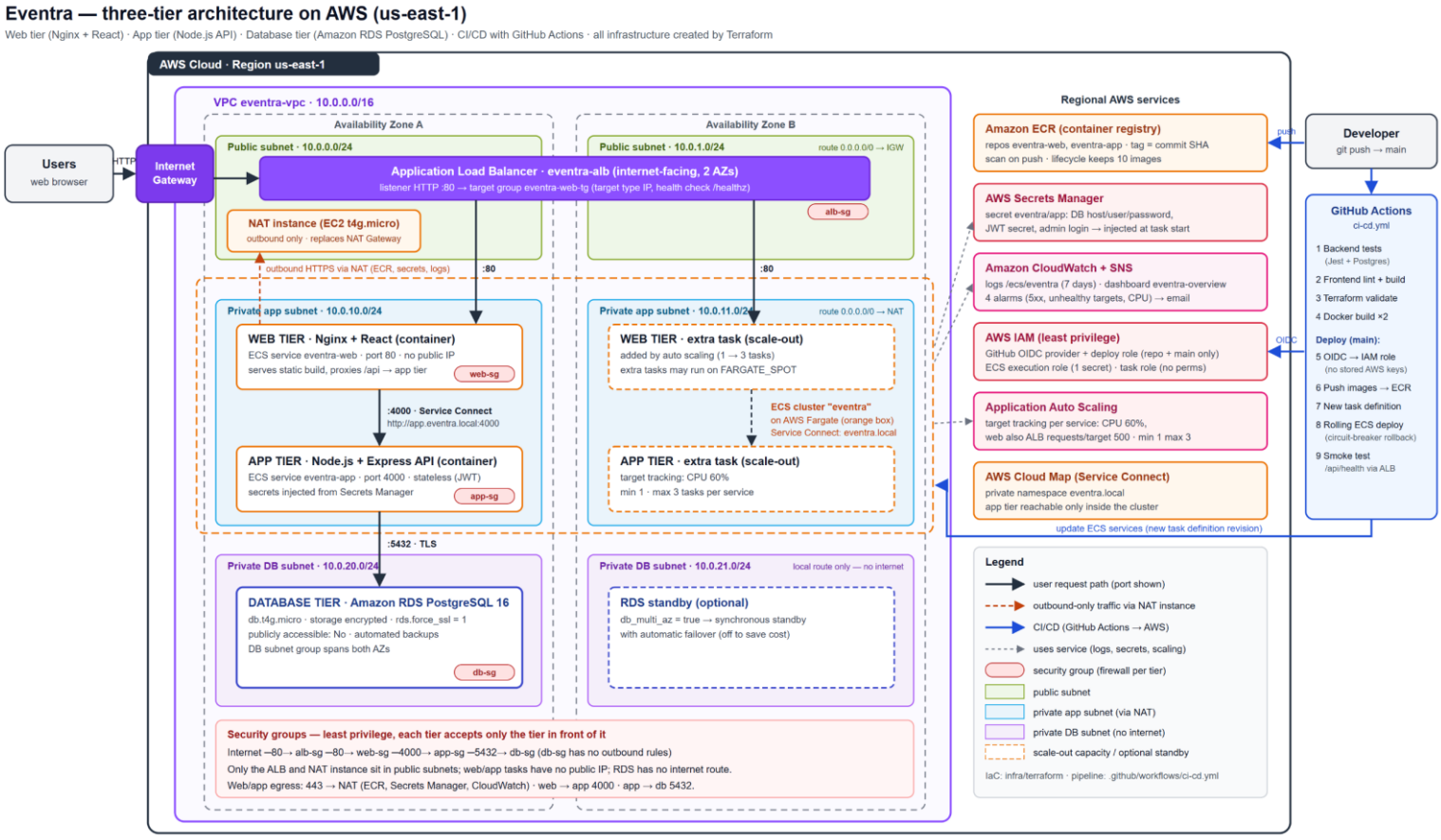


Figure 1: Eventra architecture on AWS: network, tiers, security groups and CI/CD flow

A request from the internet enters through the Internet Gateway and reaches an internet-facing Application Load Balancer placed in two public subnets. The load balancer forwards it to the web tasks (Nginx serving the React build) running in private subnets. Nginx passes /api calls through ECS Service Connect to the application tasks (Node.js/Express), which connect to an Amazon RDS PostgreSQL database in separate database subnets. The application tier is not registered with the load balancer, so it can only be reached through the web tier, as the brief requires.

The VPC uses the range 10.0.0.0/16 and is split into three layers of subnets across two Availability Zones (Table 1).

Table 1: Subnet layout and routing

Subnet	CIDR (AZ a / AZ b)	Route	Contains
Public	10.0.0.0/24, 10.0.1.0/24	0.0.0.0/0 to Internet Gateway	ALB, NAT instance
Private app	10.0.10.0/24, 10.0.11.0/24	0.0.0.0/0 to NAT instance	Web and app tasks (no public IP)
Private DB	10.0.20.0/24, 10.0.21.0/24	Local route only	RDS PostgreSQL

The private tasks only need outbound HTTPS to pull images from ECR, read Secrets Manager and send logs to CloudWatch. The NAT instance provides this without allowing any inbound connection. The database subnets have no internet route at all.

The main design decisions were:

- Two Availability Zones with three subnet layers (public, private app, private database) for segmentation and resilience.
- ECS Fargate instead of EC2 instances, so there are no servers to patch and we get per-task billing, built-in auto scaling and rolling deployments.
- A stateless application (JWT authentication, no sessions or local files), so any task can serve any request.
- A NAT instance (t4g.micro) instead of a managed NAT Gateway, which costs about $6 a month instead of about $33. The availability trade-off is covered in Section 7.
- Terraform for all infrastructure. The pipeline only changes the image version, so the deployed environment does not drift from the code.

3. Services Used and Their Role in Each Tier

Table 2: AWS services and their role

Tier / area	AWS service	Role in Eventra
Network	VPC, 6 subnets in 2 AZs, Internet Gateway, 3 route tables, NAT instance	Public subnets for the ALB and NAT; private app subnets for containers; private DB subnets with no internet route
Web tier	Application Load Balancer, target group, ECS Fargate service eventra-web	Public entry point on HTTP port 80; Nginx serves the React build and proxies /api
Application tier	ECS Fargate service eventra-app, ECS Service Connect	REST API (validation, business rules, JWT login, CSV export) on port 4000, reachable only as app.eventra.local
Database tier	Amazon RDS for PostgreSQL 16 (db.t4g.micro)	Stores events, registrations and admins; encrypted, TLS only, not publicly accessible

Tier / area	AWS service	Role in Eventra
Registry	Amazon ECR (eventra-web, eventra-app)	Stores images tagged with the Git commit SHA; scan on push
Scaling	Application Auto Scaling	Target tracking on CPU (60%) and ALB requests per target (500); 1 to 3 tasks per service
Security	Security groups, IAM, Secrets Manager, GitHub OIDC	Per-tier firewalls, least-privilege roles, secrets injected at runtime, keyless CI/CD
Operations	CloudWatch Logs, dashboard, alarms, SNS	Central container and database logs, metrics dashboard and alerts

Additional cloud-native enhancements. Besides the required parts we implemented three of the optional items: infrastructure as code with Terraform, centralized logging and monitoring with CloudWatch, and a cost estimate. Using the AWS Pricing Calculator, the estimated cost is about $68 per month (roughly $2.25 per day) in us-east-1 with low traffic, mostly the load balancer (about $18), Fargate tasks (about $18), RDS (about $14) and public IPv4 addresses (about $11). The calculator estimate is at https://calculator.aws/#/estimate?id=533269af66c81552ce9ac5c38996bdeb38381b98. We applied these cost optimizations: (1) a NAT instance instead of a NAT Gateway, saving about $27 per month; (2) the smallest Fargate task size with one task per tier at rest, automatic scale-in and Fargate Spot for extra tasks; (3) Graviton (t4g) instances for NAT and RDS; and (4) 7-day log retention, an ECR lifecycle policy, and running terraform destroy when the environment is idle.

4. Containerization and CI/CD Pipeline Design

The web image is built in two stages: a node:22-alpine stage runs the Vite production build, and an nginx:1.27-alpine stage serves the static files (about 75 MB). Nginx is configured with gzip, long caching for hashed assets and a fallback to index.html for the single-page app. The /api upstream is set through the BACKEND_URL variable, so the same image works in docker-compose and on ECS. The application image is based on node:22-alpine with production dependencies only, runs as the non-root node user, and has a HEALTHCHECK on /api/health. The docker-compose.yml starts all three tiers locally in health-check order, and only the web service publishes a port.

The pipeline is defined in .github/workflows/ci-cd.yml and runs on every push and pull request (Table 3).

Table 3: CI/CD pipeline stages

Stage	What it does
1. Backend tests	Runs 78 Jest and Supertest tests against a PostgreSQL 16 service container, with coverage
2. Frontend	ESLint and the Vite production build
3. Terraform validate	terraform fmt -check, init and validate
4. Docker build	Builds both images with Buildx layer caching to confirm the Dockerfiles work
5. Deploy (main only)	Logs in to AWS with OIDC, builds and pushes both images to ECR (tag = commit SHA), downloads the live task definitions, inserts the new image, updates eventra-app and then eventra-web with a rolling update, waits for the services to become stable, and finally smoke-tests /api/health through the load balancer

ECS deployment circuit breakers roll back automatically if a new version never becomes healthy, and a concurrency group stops two deployments from overlapping. The deploy job is skipped (not failed) until the repository variable AWS_DEPLOY_ENABLED is set.

5. Security Considerations

Table 4: Security controls

Control	Implementation
Security groups per tier	alb-sg allows port 80 from the internet; web-sg allows 80 from alb-sg only; app-sg allows 4000 from web-sg only; db-sg allows 5432 from app-sg only and has no outbound rules. Rules refer to security groups rather than IP ranges.
Network segmentation	Only the ALB and NAT instance are public. Tasks have no public IPs, and RDS is not publicly accessible and sits in subnets with no internet route.
Least privilege	The ECS execution role can read only the eventra/app secret and the task role has no permissions. The GitHub deploy role trusts only this repository on main, and can only push to the two ECR repositories, update the two ECS services and pass the two ECS roles.
Secrets management	Terraform generates the database, JWT and admin passwords and stores them in Secrets Manager; ECS injects them as environment variables at start-up. No secret is stored in the code, images or Git (.env is git-ignored). The pipeline uses short-lived OIDC credentials instead of stored AWS keys.
Data protection	RDS storage encryption, mandatory TLS to the database (rds.force_ssl = 1), bcrypt password hashing, parameterized SQL and CSV-injection protection.
Application hardening	helmet and Nginx security headers, CORS allow-list, rate limiting on login and registration, Zod input validation, JWT expiry, non-root containers, ECR image scanning, and IMDSv2 with no SSH on the NAT instance.

6. Testing Results and Automated Deployment

Table 5: Test results

Test	What it covered	Result
Backend (Jest + Supertest)	Health check and DB-down 503, event listing and filters, registration, duplicate email, full event, concurrent overbooking, admin login, CRUD, CSV export, rate limits	78 / 78 passed (88% statement coverage)
API end-to-end via Nginx	Full flow on docker compose, including checks that the DB and API ports are not exposed	27 / 27 passed
Browser UI (headless Chrome)	Filters, registration, confirmation, admin login, create/delete event and CSV export on desktop and phone sizes	30 / 30 passed (after 2 fix rounds)
Cloud readiness	Two app replicas using one JWT, health 503 when DB is down, data kept after restart	3 / 3 passed
GitHub Actions CI/CD	Tests, lint and build, Terraform validate, Docker builds, push to ECR, rolling ECS deploy, smoke test	Successful deploy on main (Appendix A)
Live AWS deployment	The same 30 browser checks run against the load balancer address	30 / 30 passed on AWS

Testing found one serious bug and two layout problems, which we fixed. The bug was a React effect that returned the Promise from window.scrollTo(), which blanked the page on navigation in current Chrome. The first deployment to AWS also exposed four configuration problems: the ECS service-linked role was not ready on a new account, the free account plan rejected the t4g.nano instance type, GitHub's OIDC subject now includes the owner and repository IDs (so we extended the IAM trust policy), and one repository variable had a wrong value. All four were fixed.

We also ran a load test against the live site (about 25,000 requests, 1 failed). Requests per target went up, and auto scaling increased the web service from 1 to 3 tasks without any manual action (Figure 2). About 20 minutes after the traffic stopped it went back to 1 task. A code push was deployed through the pipeline during the same test, which showed a rolling update working under load.

```
# ECS web service task count during the load test (sampled from AWS)
# time (UTC) | desired | running | ALB healthy targets
19:30:02 | 1 | 1 | 1
19:31:40 | 1 | 1 | 1
19:32:50 | 1 | 1 | 1
19:33:54 | 3 | 1 | 1
19:35:07 | 3 | 3 | 3
19:36:13 | 3 | 3 | 3
19:37:05 | 3 | 3 | 5
19:38:33 | 3 | 3 | 3
19:39:37 | 3 | 3 | 3
19:41:22 | 3 | 3 | 3
19:42:42 | 3 | 3 | 3
19:43:29 | 3 | 3 | 3
19:44:20 | 3 | 3 | 3
19:45:06 | 3 | 3 | 3
19:45:49 | 3 | 3 | 3
19:46:33 | 3 | 3 | 3
19:47:24 | 3 | 3 | 3
19:48:08 | 3 | 3 | 3
19:48:52 | 3 | 3 | 3
19:49:44 | 3 | 3 | 3
19:50:30 | 3 | 3 | 3
19:51:13 | 3 | 3 | 3
```

Figure 2: Web tasks scaling from 1 to 3 during the load test (the value 5 appears during a rolling deploy)

7. Non-Functional Requirements Justification

Table 6: NFRs mapped to design decisions

NFR	How the design addresses it	Limitation and improvement
Scalability	Stateless containers; the ALB spreads traffic over a target group; both tiers can scale horizontally; row locking and a unique constraint keep data correct with many replicas; queries are indexed and paginated.	Single RDS instance. Add read replicas or Aurora, and RDS Proxy.
Availability / Reliability	Two AZs; ALB across AZs; health checks at the ALB, container and /api/health (checks the DB) level; ECS replaces failed tasks; circuit breaker with rollback; managed RDS backups; graceful shutdown.	To save cost we used 1 task per tier, single-AZ RDS and one NAT instance. Improve with min_tasks = 2, db_multi_az = true and a NAT Gateway per AZ.
Elasticity	Target-tracking auto scaling on CPU (60%) and ALB requests per target, 1 to 3 tasks, fast scale-out (60 s) and automatic scale-in (180 s); no pre-provisioned servers with Fargate; RDS storage autoscaling.	DB compute is fixed. Aurora Serverless v2 would fix this.
Security	Segmented subnets, chained least-privilege security groups, Secrets Manager, OIDC for CI/CD, TLS to the database, encryption at rest, application hardening (Section 5).	HTTP only at the ALB. Add an ACM certificate with an HTTPS listener and AWS WAF.

NFR	How the design addresses it	Limitation and improvement
Maintainability / Deployability	The same containers run locally, in CI and on ECS; every push runs test, build, push, rolling deploy and smoke test; images are tagged with the commit SHA; infrastructure is one Terraform command.	No staging environment. Add a Terraform workspace per environment with an approval step.
Performance / Latency	Nginx gzip and long caching for hashed assets; code-split admin bundle; indexed SQL; all tiers in one region and VPC; scaling starts before the tasks are saturated.	No CDN or cache. Add CloudFront and ElastiCache.
Cost-Efficiency	About $68 per month compared with about $127 for a standard high-availability layout: NAT instance, smallest Fargate size, Spot for extra tasks, Graviton for NAT and RDS, short log retention.	The savings reduce availability (see above).
Portability	OCI images, environment-variable configuration, standard PostgreSQL and a docker-compose file that runs anywhere. Only Terraform and the deploy job are AWS-specific.	Add Kubernetes manifests or Helm charts for provider-neutral deployment.

The NFRs that our solution does not fully meet are availability (single NAT instance, single-AZ database and one task per tier at rest) and security (no HTTPS at the load balancer). Both were deliberate cost decisions, and each can be improved with a small Terraform or configuration change as listed above.

Appendix A: Deployment Evidence

The screenshots below are from the live deployment in AWS (us-east-1), GitHub and the browser. Figures marked “CLI output” are unmodified AWS CLI output.

A.1 Network / VPC configuration and subnets

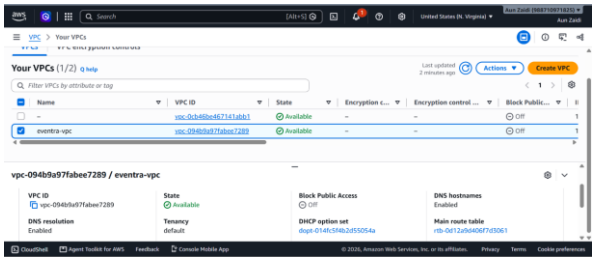


Figure 3: VPC eventra-vpc (10.0.0.0/16)

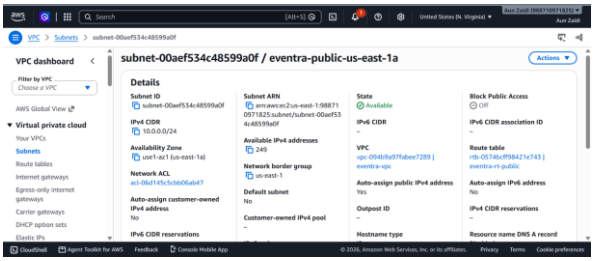


Figure 4: Public subnet, us-east-1a (10.0.0.0/24)

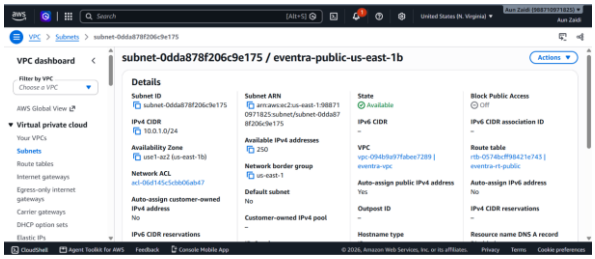


Figure 5: Public subnet, us-east-1b (10.0.1.0/24)

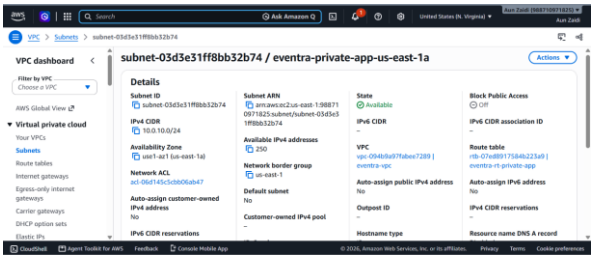


Figure 6: Private app subnet, us-east-1a (10.0.10.0/24)

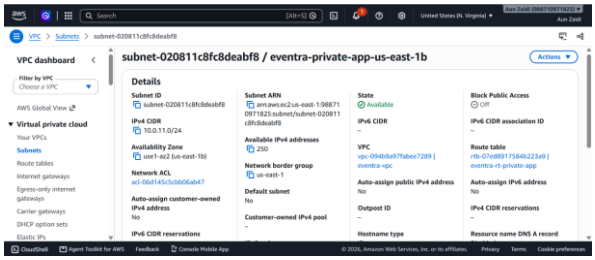


Figure 7: Private app subnet, us-east-1b (10.0.11.0/24)

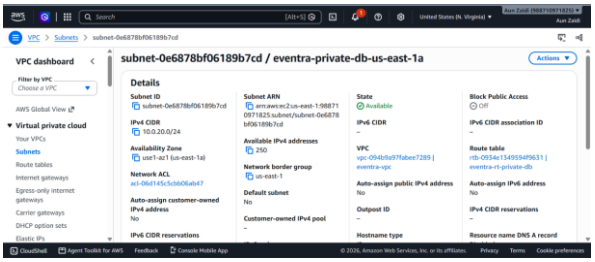


Figure 8: Private DB subnet, us-east-1a (10.0.20.0/24)

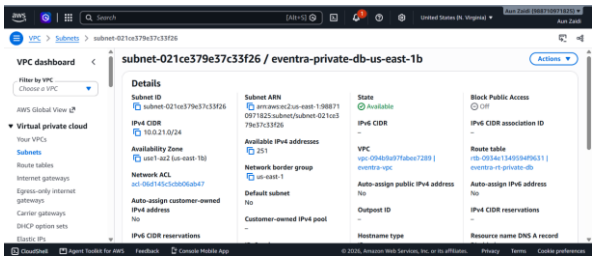


Figure 9: Private DB subnet, us-east-1b (10.0.21.0/24)

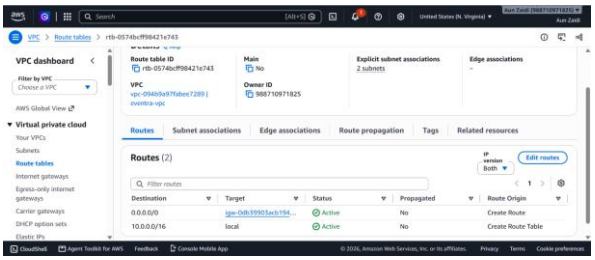


Figure 10: Public route table: 0.0.0.0/0 to Internet Gateway

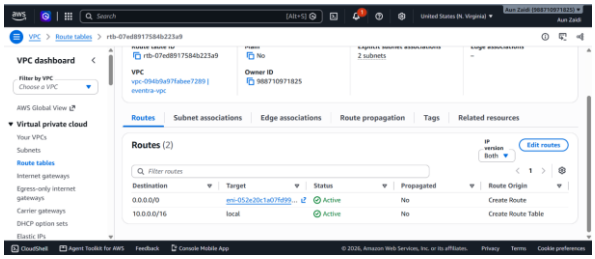


Figure 11: Private app route table: 0.0.0.0/0 to NAT instance

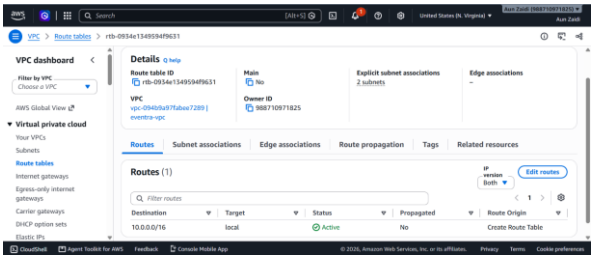


Figure 12: Private DB route table: local route only

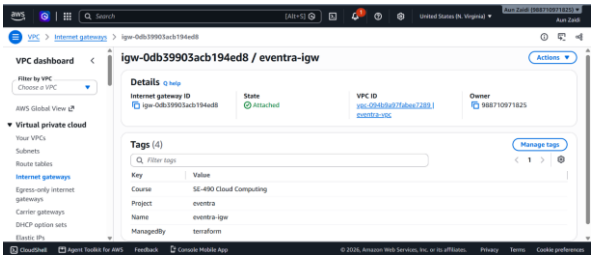


Figure 13: Internet Gateway attached to the VPC

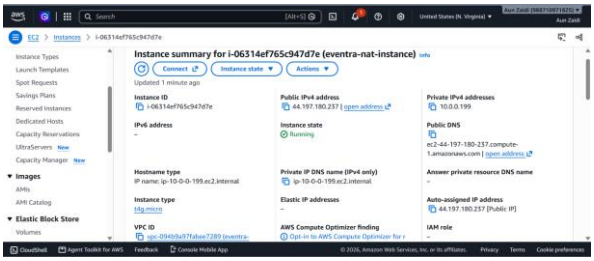


Figure 14: NAT instance (t4g.micro) in the public subnet

A.2 Compute instances / container hosts

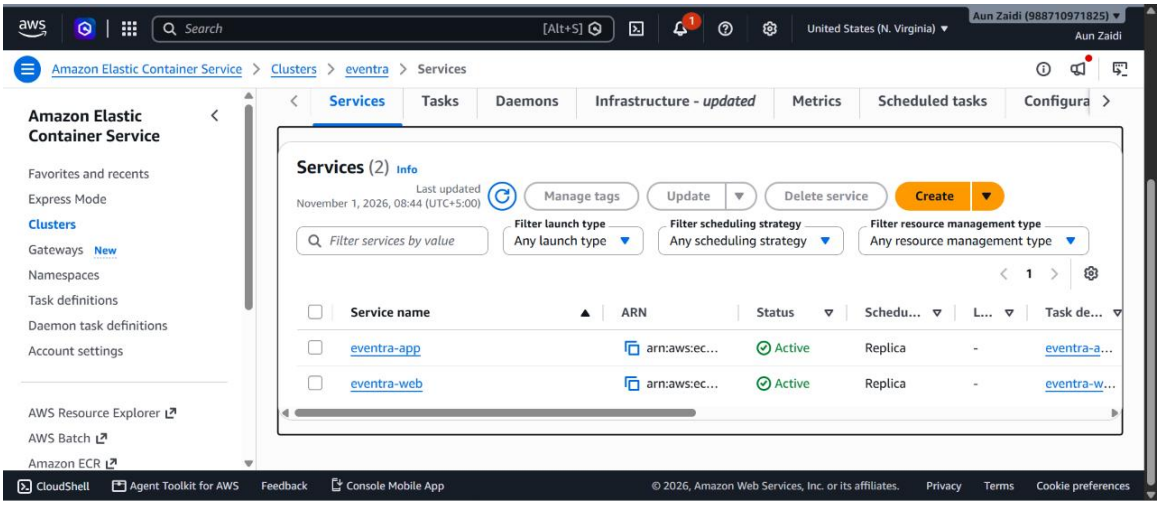


Figure 15: ECS cluster with the web and app services

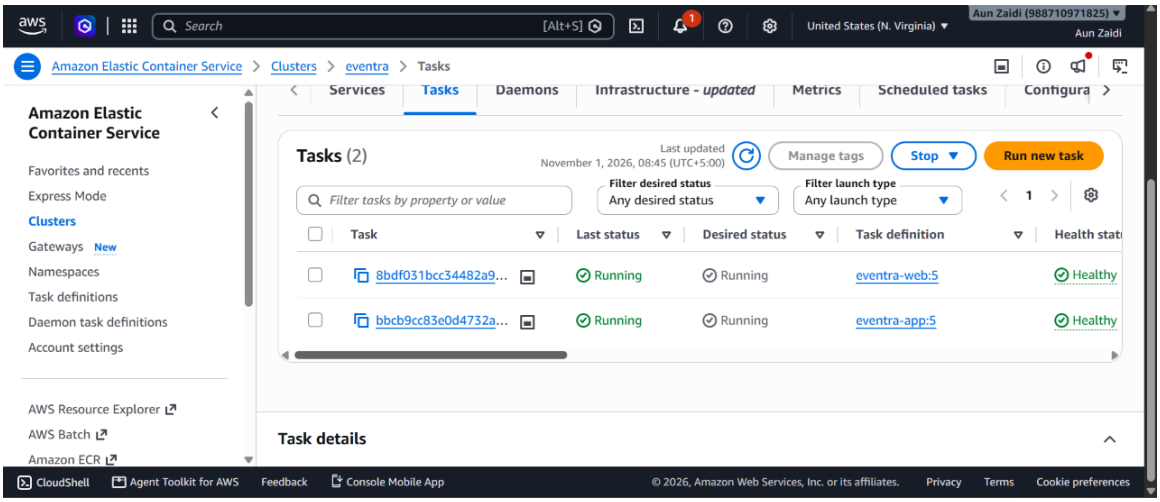


Figure 16: Running, healthy Fargate tasks

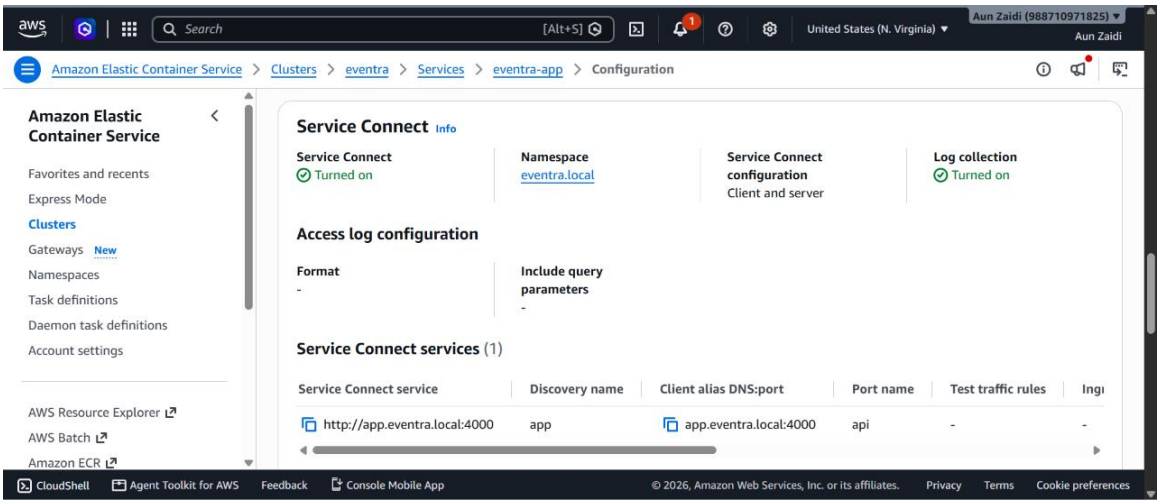


Figure 17: Service Connect on eventra-app (app.eventra.local:4000)

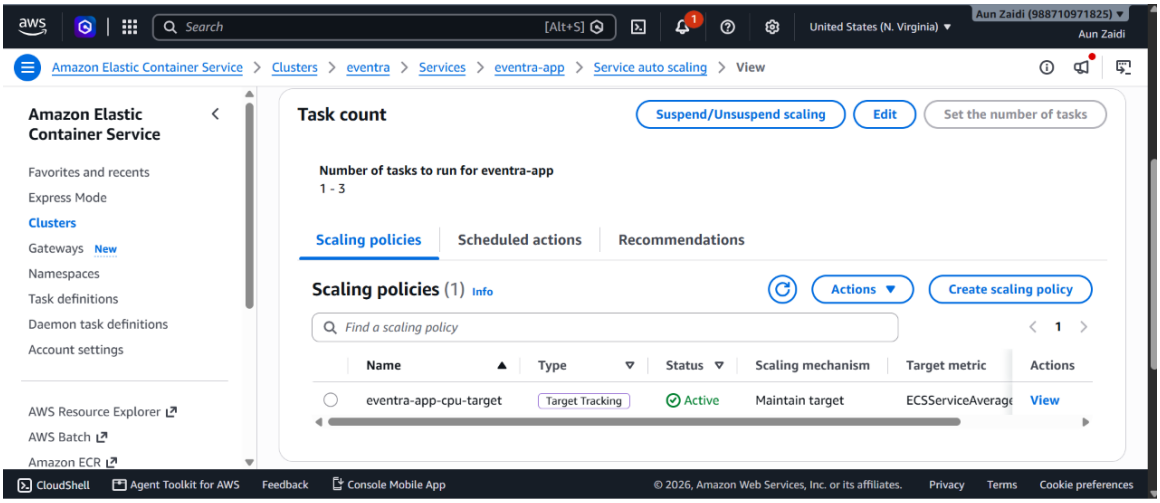


Figure 18: Service auto scaling on eventra-app (1 to 3 tasks, target tracking)

```
# AWS record of ECS auto scaling activity on eventra-web
2026-11-01T00:56:03.658000+05:00        Successful      monitor alarm TargetTracking-service/eventra/eventra-web-AlarmLow-28a2a1cb-1b27-4e2b-a48b-
bb993faffe5b in state ALARM triggered policy eventra-web-requests-target        Setting desired count to 1.
2026-11-01T00:52:03.687000+05:00        Successful      monitor alarm TargetTracking-service/eventra/eventra-web-AlarmLow-28a2a1cb-1b27-4e2b-a48b-
bb993faffe5b in state ALARM triggered policy eventra-web-requests-target        Setting desired count to 2.
2026-11-01T00:33:16.003000+05:00        Successful      monitor alarm TargetTracking-service/eventra/eventra-web-AlarmHigh-ce3a6080-bccd-4c3a-b319-
32105dd318f3 in state ALARM triggered policy eventra-web-requests-target        Setting desired count to 3.
```

Figure 19: AWS record of alarms raising and lowering the task count during the load test

A.3 Security groups

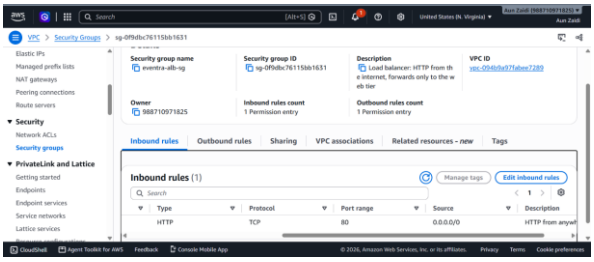


Figure 20: alb-sg inbound: HTTP 80 from the internet

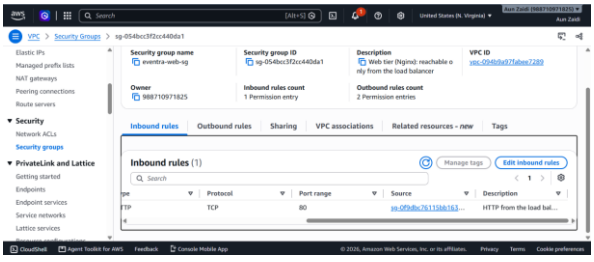


Figure 21: web-sg inbound: 80 from alb-sg only

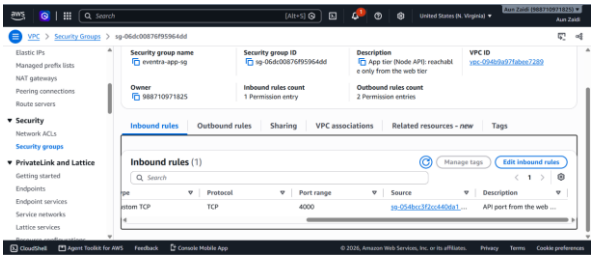


Figure 22: app-sg inbound: 4000 from web-sg only

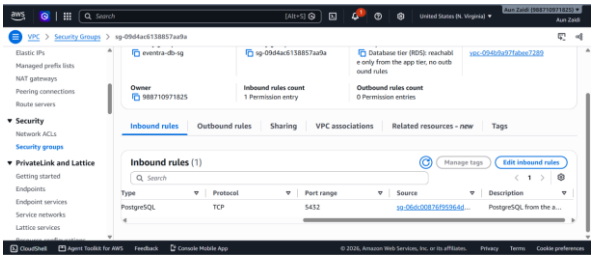


Figure 23: db-sg inbound: 5432 from app-sg only, no outbound rules

A.4 Load balancer configuration

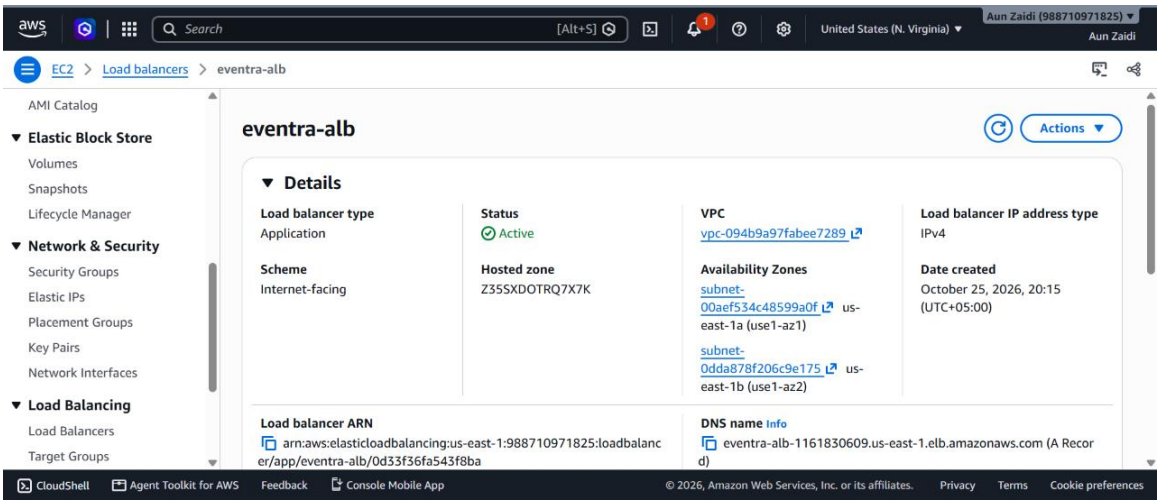


Figure 24: Internet-facing ALB in two Availability Zones

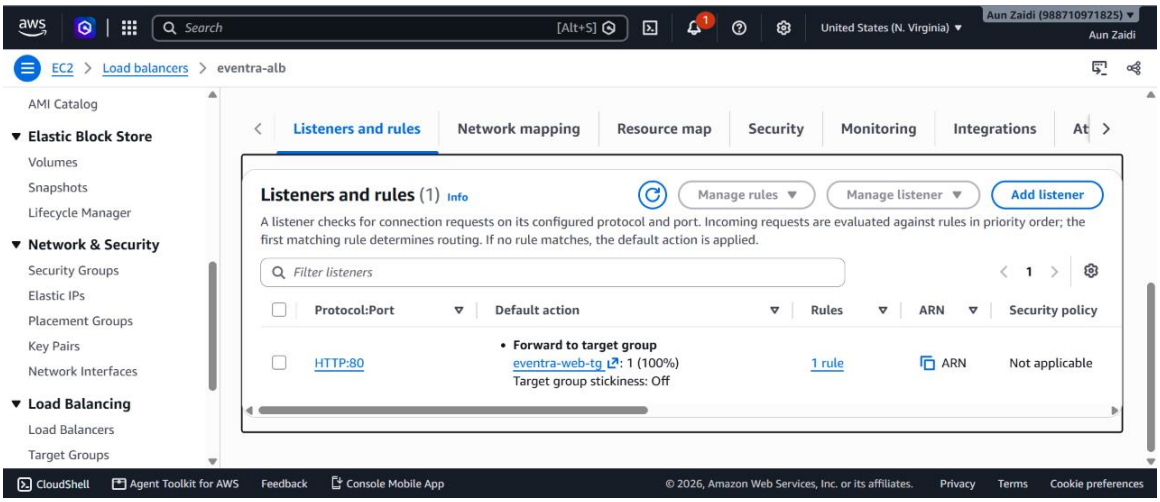


Figure 25: Listener HTTP:80 forwarding to eventra-web-tg

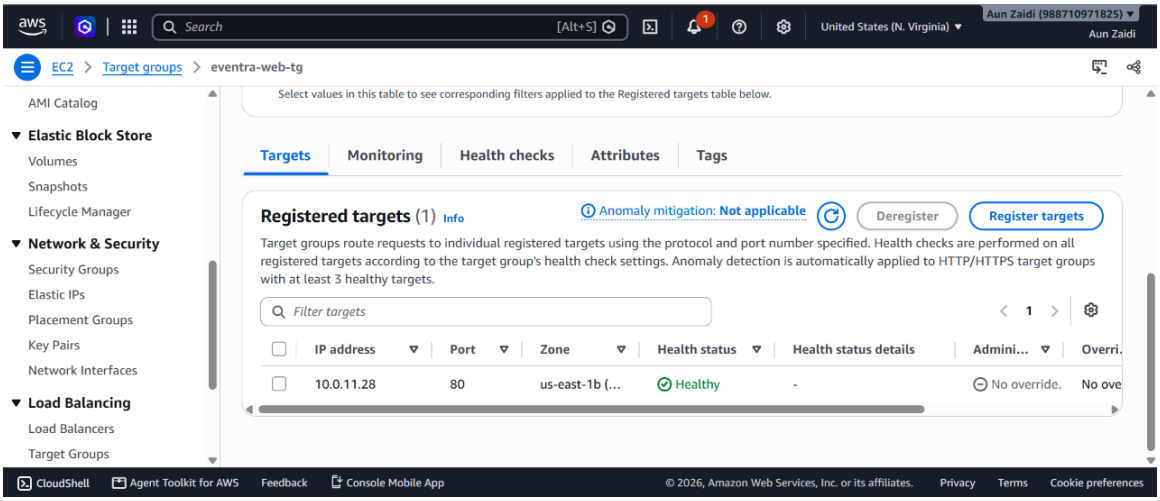


Figure 26: Target group with healthy web task targets

A.5 Database instance

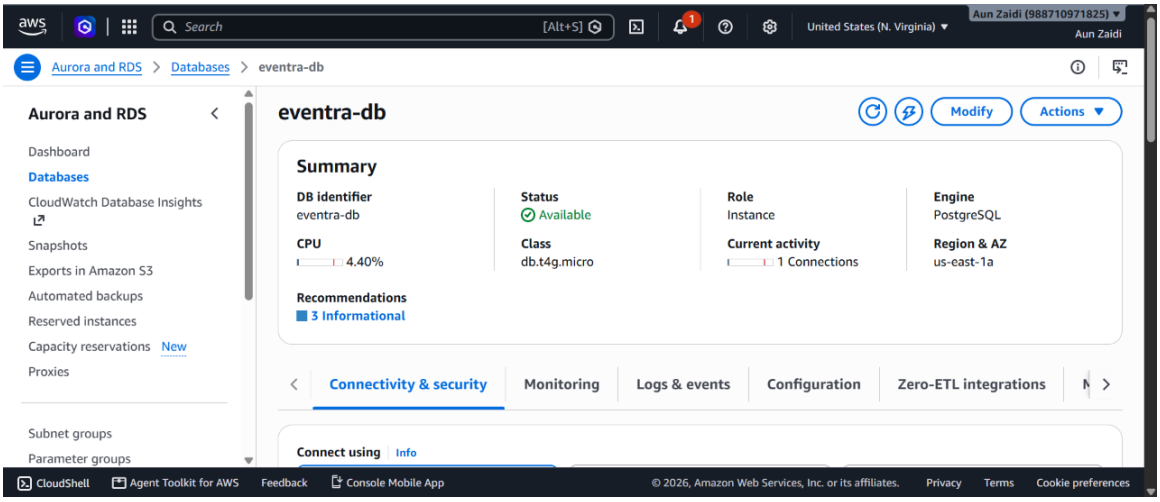


Figure 27: RDS PostgreSQL database eventra-db, status Available

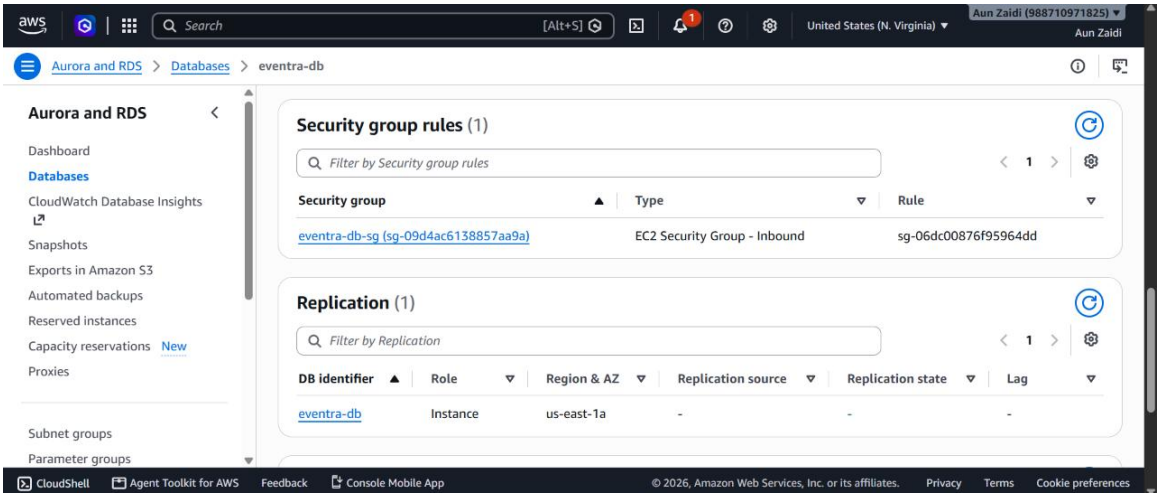


Figure 28: RDS inbound rule: only the app tier security group

```
# RDS PostgreSQL instance
# $ aws_ rds describe-db-instances --db-instance-identifier eventra-db --query DBInstances[].
{Id:DBInstanceIdentifier,Engine:Engine,Version:EngineVersion,Class:DBInstanceClass,Status:DBInstanceStatus,PubliclyAccessible:PubliclyAccessible,Mul
tiAZ:MultiAZ,Encrypted:StorageEncrypted,SubnetGroup:DBSubnetGroup.DBSubnetGroupName,SecurityGroups:VpcSecurityGroups[].VpcSecurityGroupId} --output
yaml
# captured 2026-10-31T19:55:01Z

- Class: db.t4g.micro
  Encrypted: true
  Engine: postgres
  Id: eventra-db
  MultiAZ: false
  PubliclyAccessible: false
  SecurityGroups:
  - sg-09d4ac6138857aa9a
  Status: available
  SubnetGroup: eventra-db-subnets
  Version: '16.13'
```

Figure 29: RDS PostgreSQL: publicly accessible false, storage encrypted (CLI output)

A.6 Docker image build and registry listing

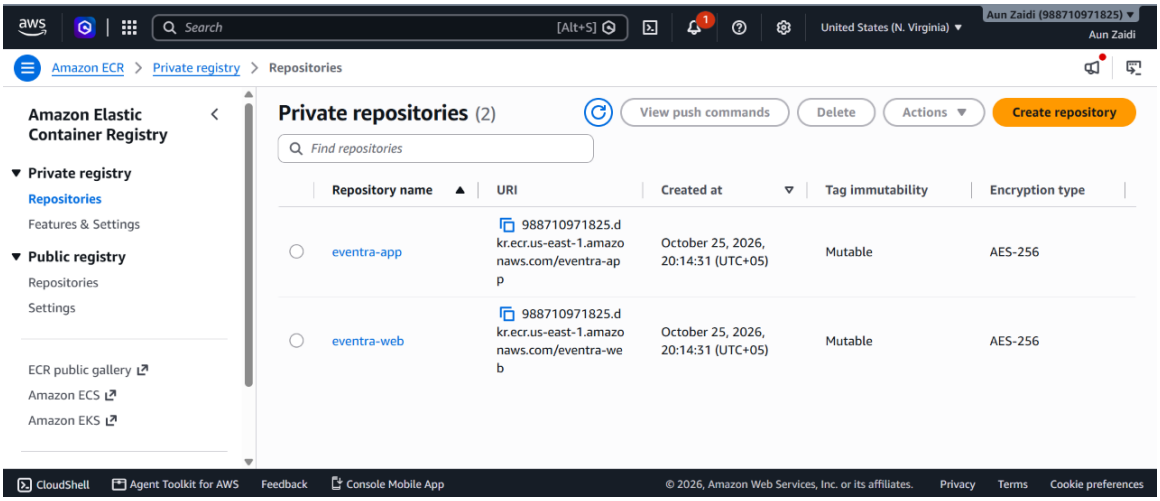


Figure 30: ECR repositories eventra-app and eventra-web

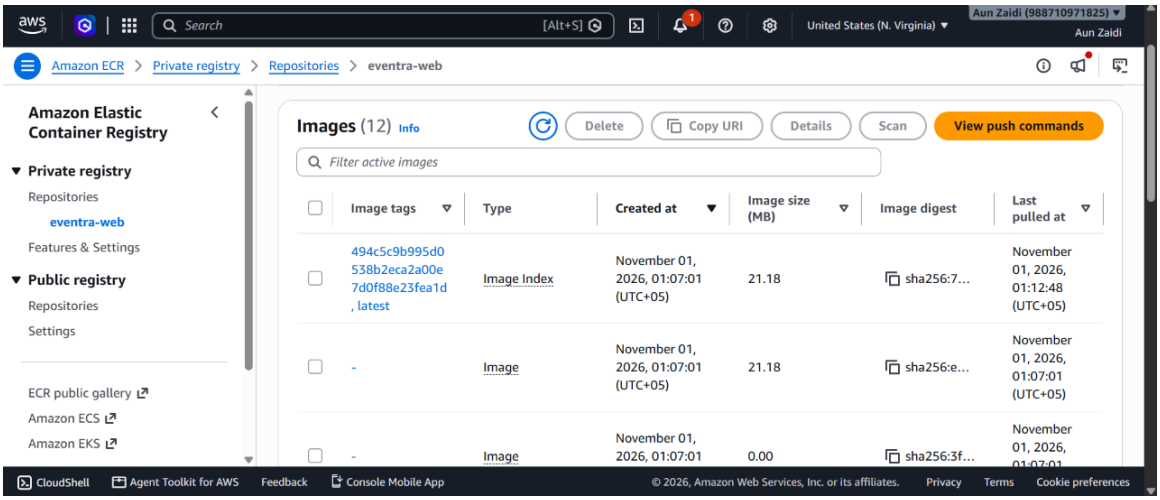


Figure 31: eventra-web images tagged with commit SHA and latest

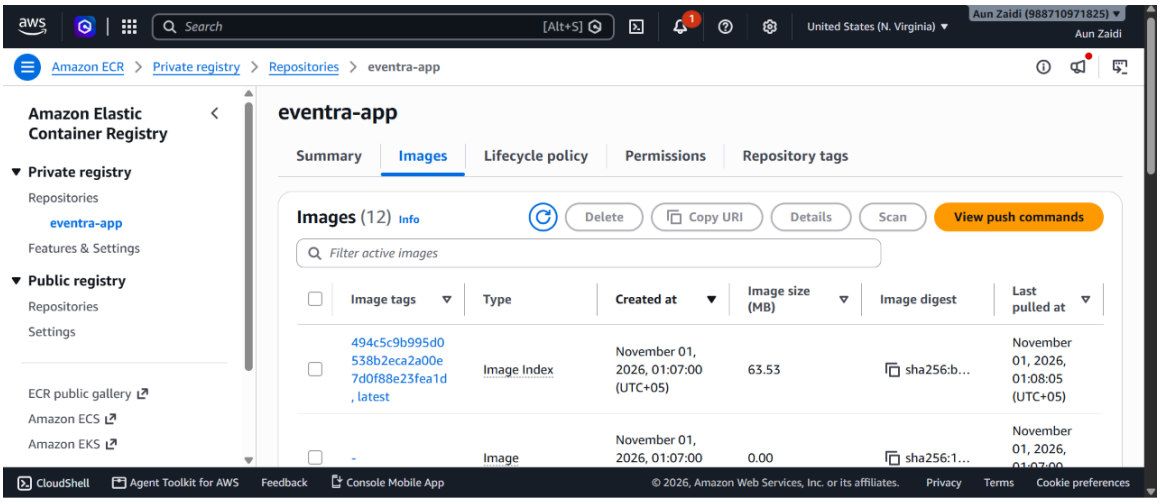


Figure 32: eventra-app images tagged with commit SHA and latest

A.7 Secrets, IAM and monitoring

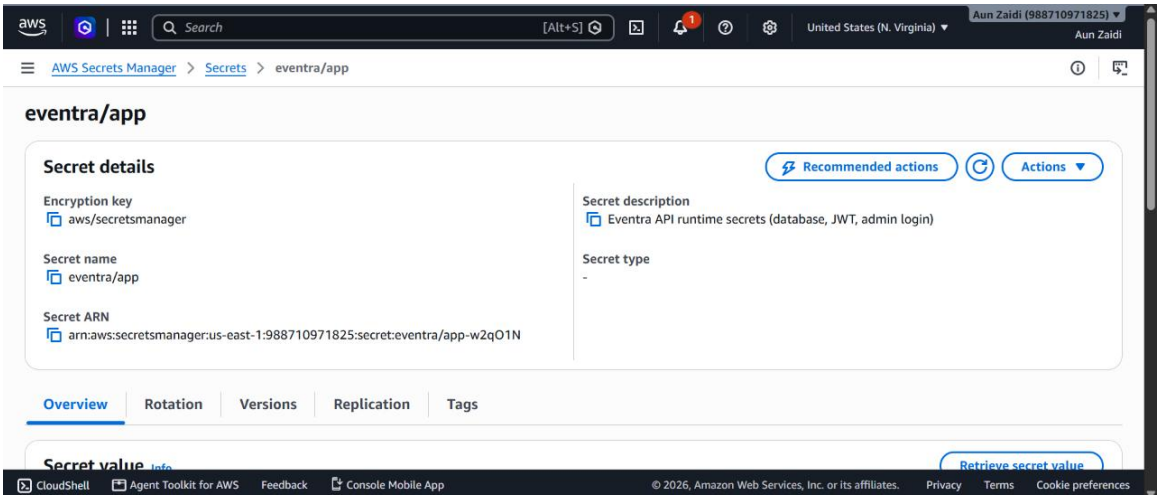


Figure 33: Secrets Manager secret eventra/app (value not shown)

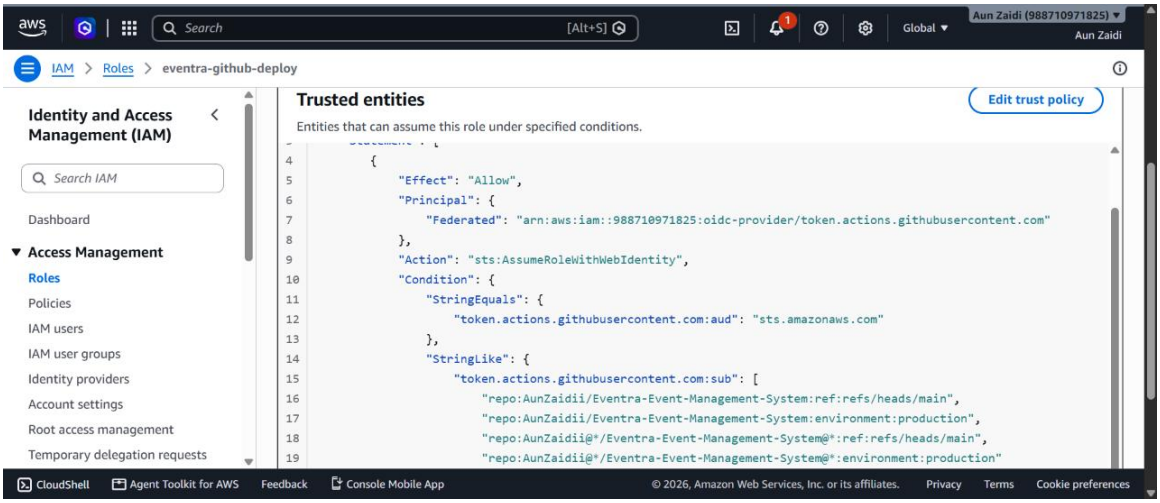


Figure 34: GitHub OIDC deploy role trust policy

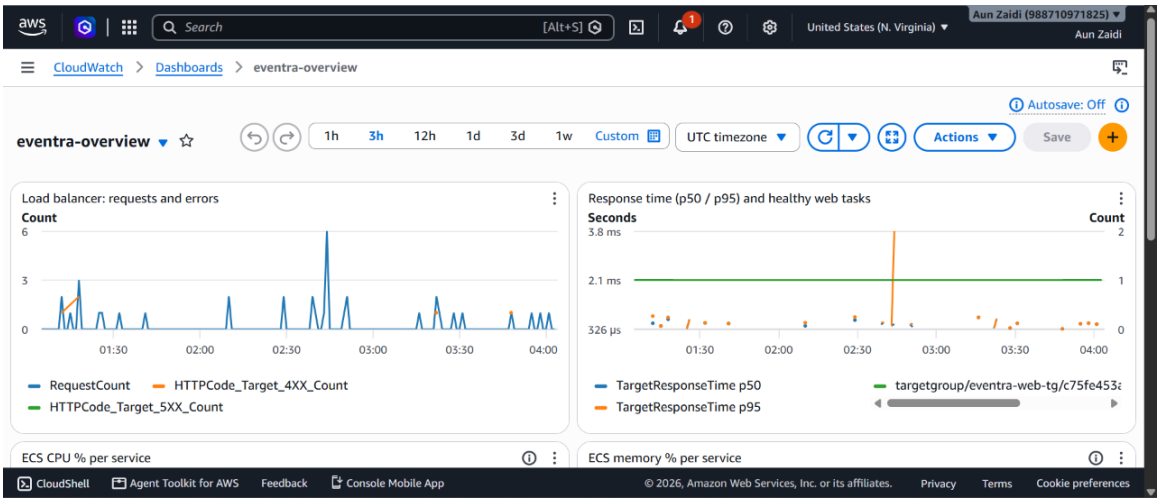


Figure 35: CloudWatch dashboard eventra-overview

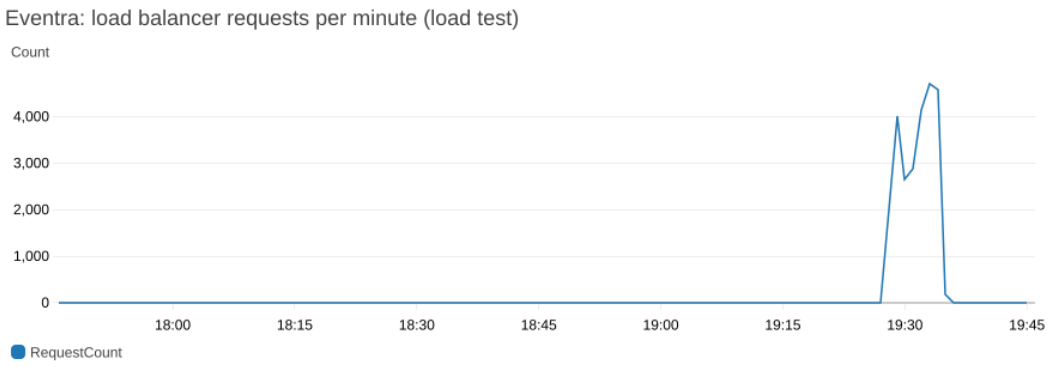


Figure 36: Load balancer requests per minute during the load test

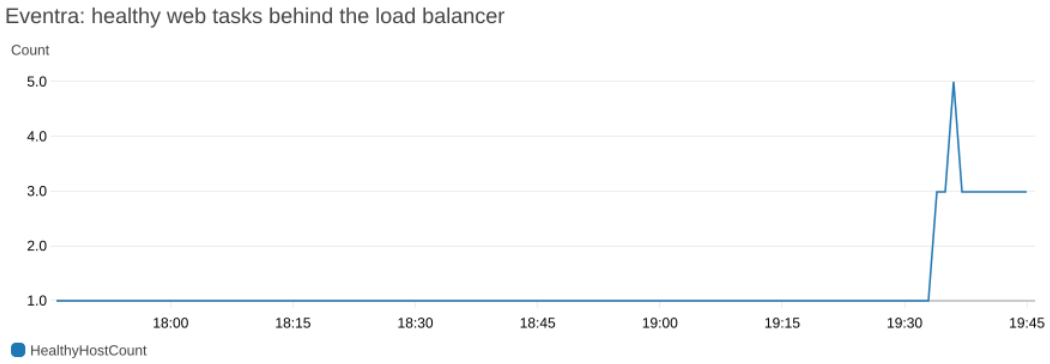


Figure 37: Healthy web tasks behind the load balancer

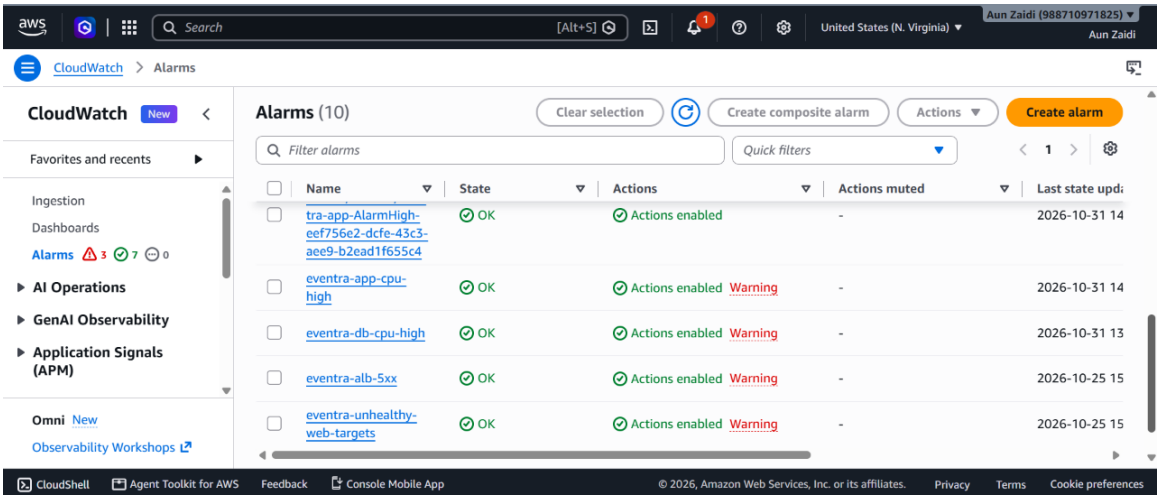


Figure 38: CloudWatch alarms

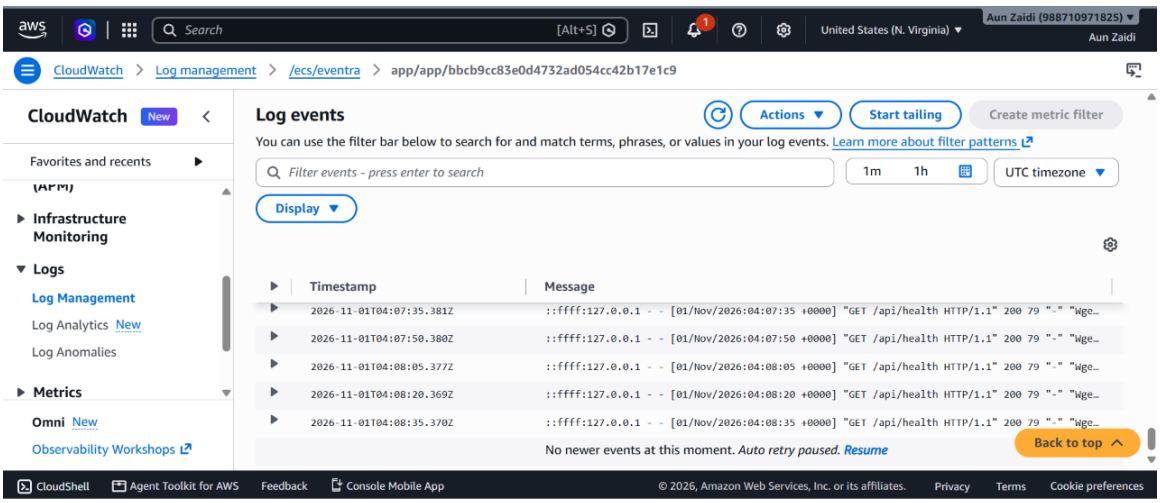


Figure 39: Centralized container logs (/ecs/eventra)

A.8 CI/CD pipeline run

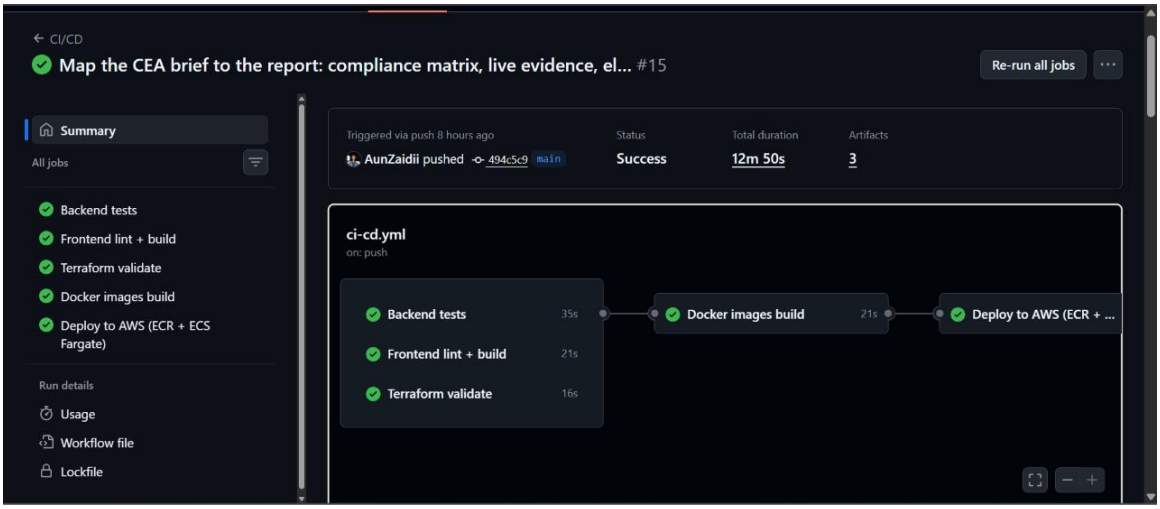


Figure 40: GitHub Actions run with all jobs successful, including the AWS deploy

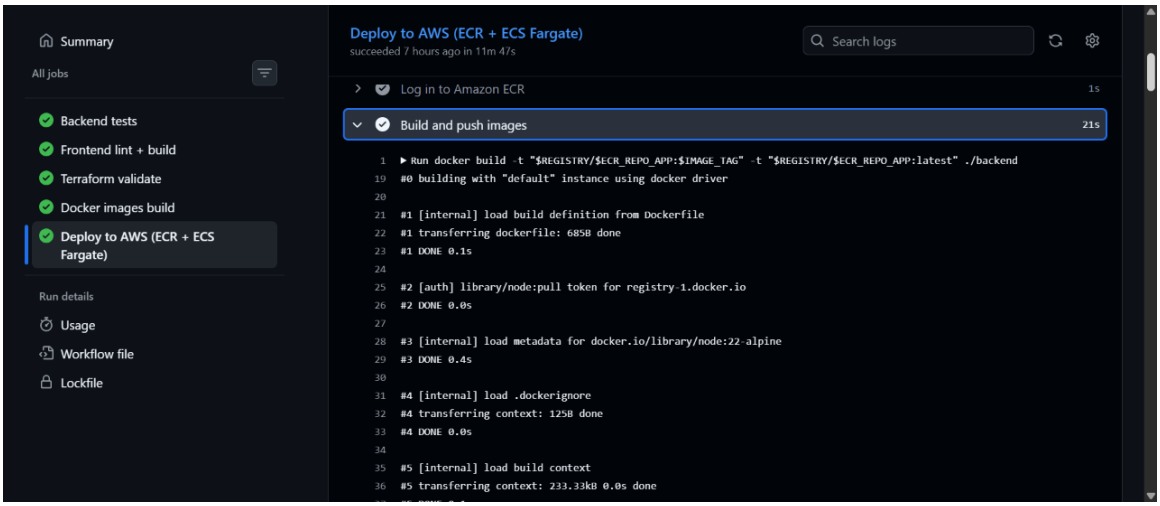


Figure 41: Deploy job: Docker images built and pushed to ECR

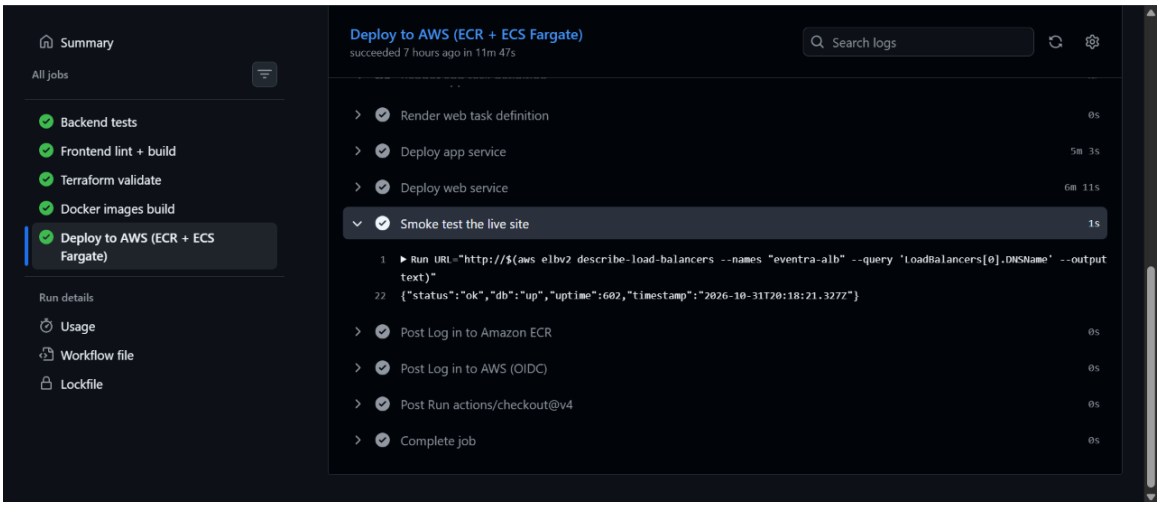


Figure 42: Smoke test: live site reports status ok, database up

A.9 Cost estimate

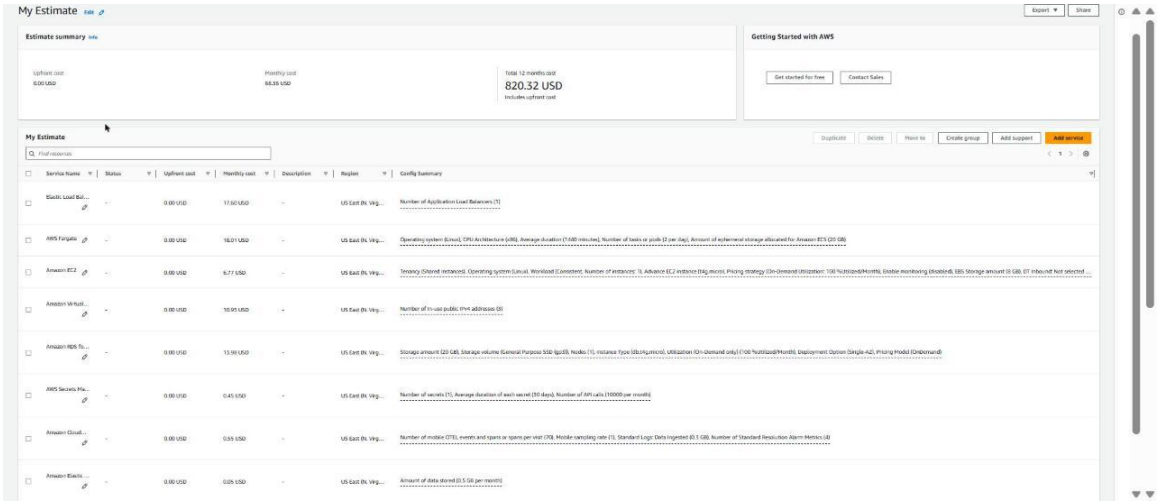


Figure 43: AWS Pricing Calculator estimate

A.10 Running application in the browser

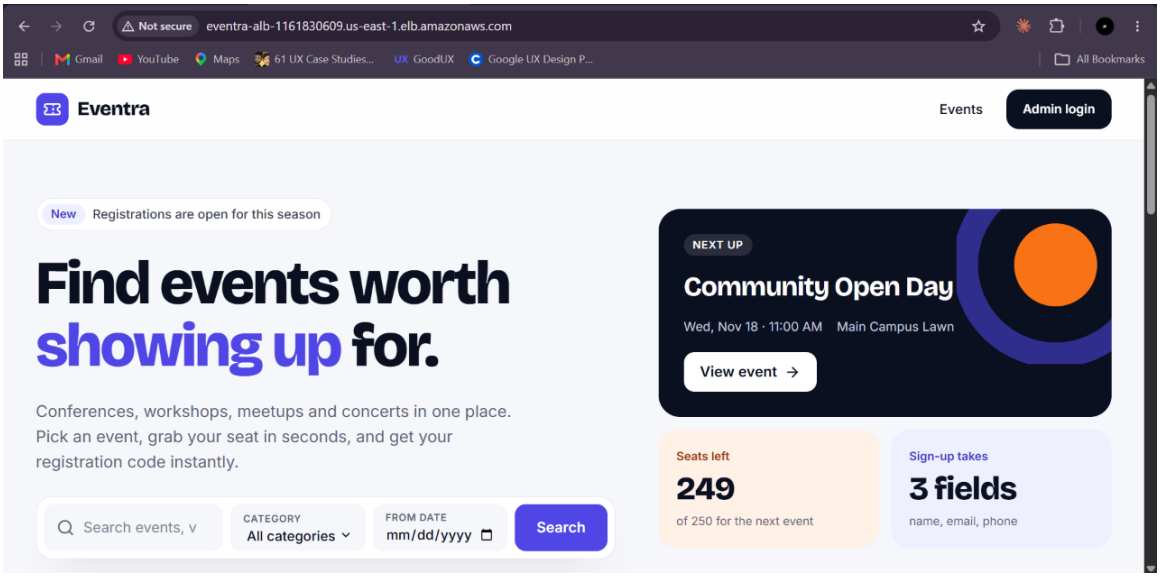


Figure 44: Live site opened in a browser using the load balancer address

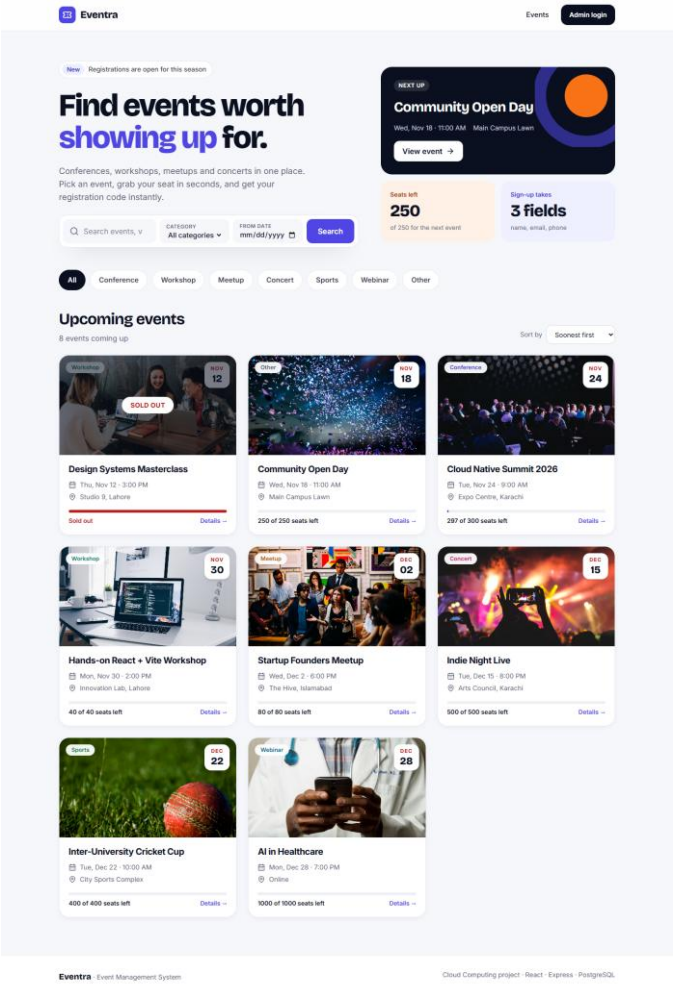


Figure 45: Homepage with upcoming events

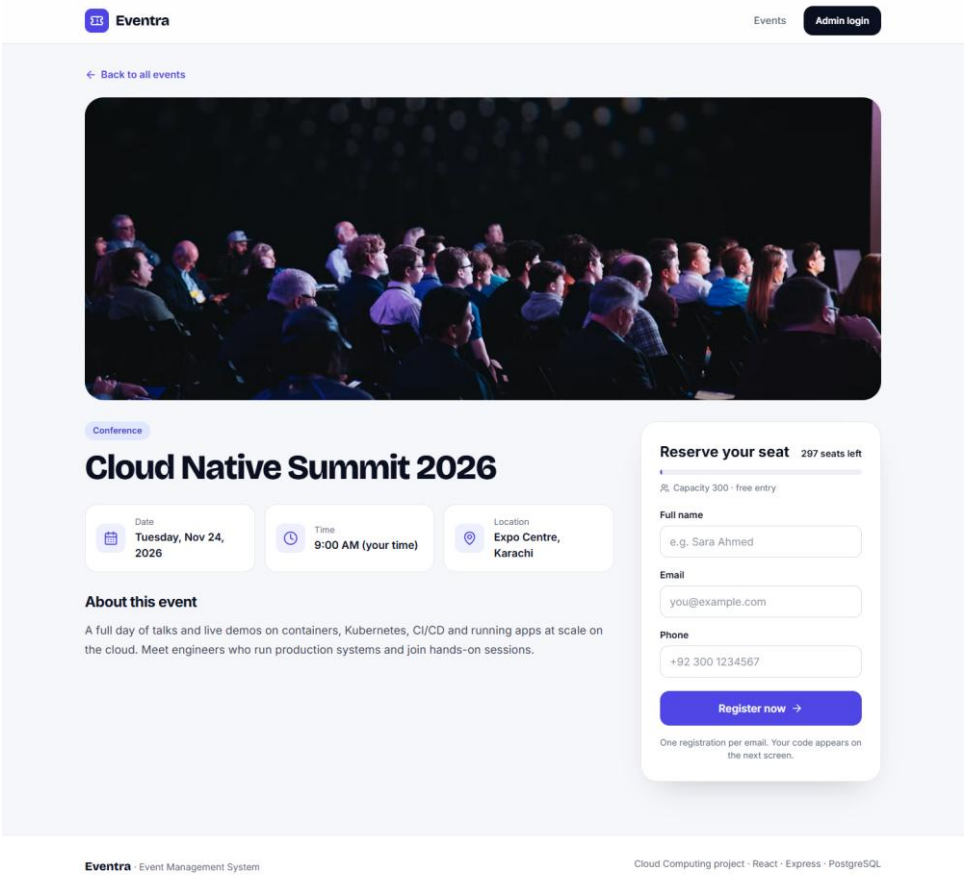


Figure 46: Event details with registration form

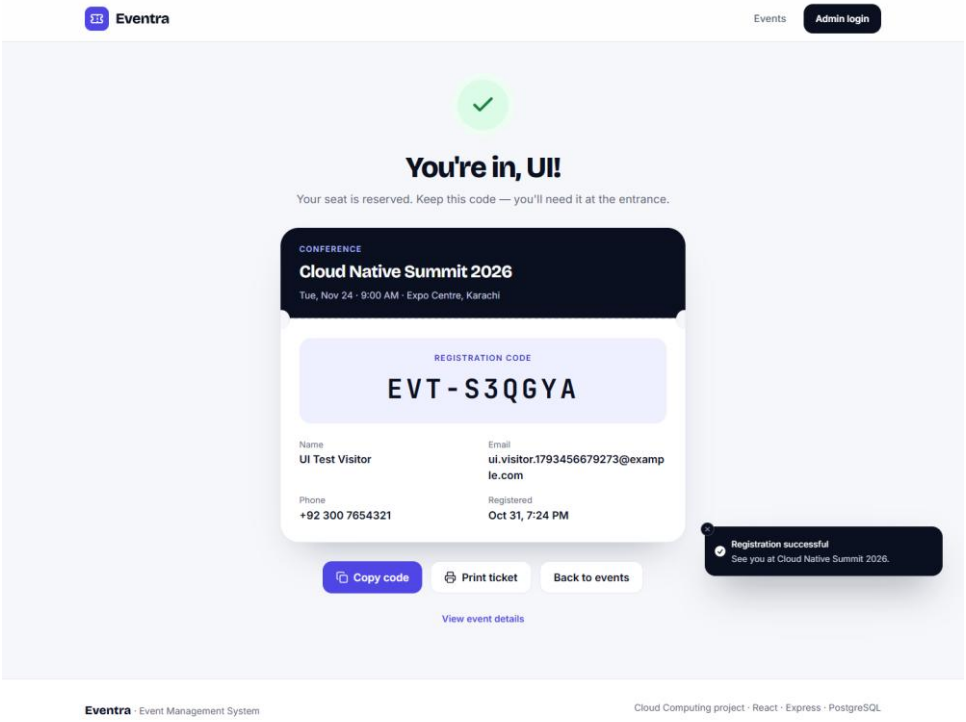


Figure 47: Registration confirmation with unique code

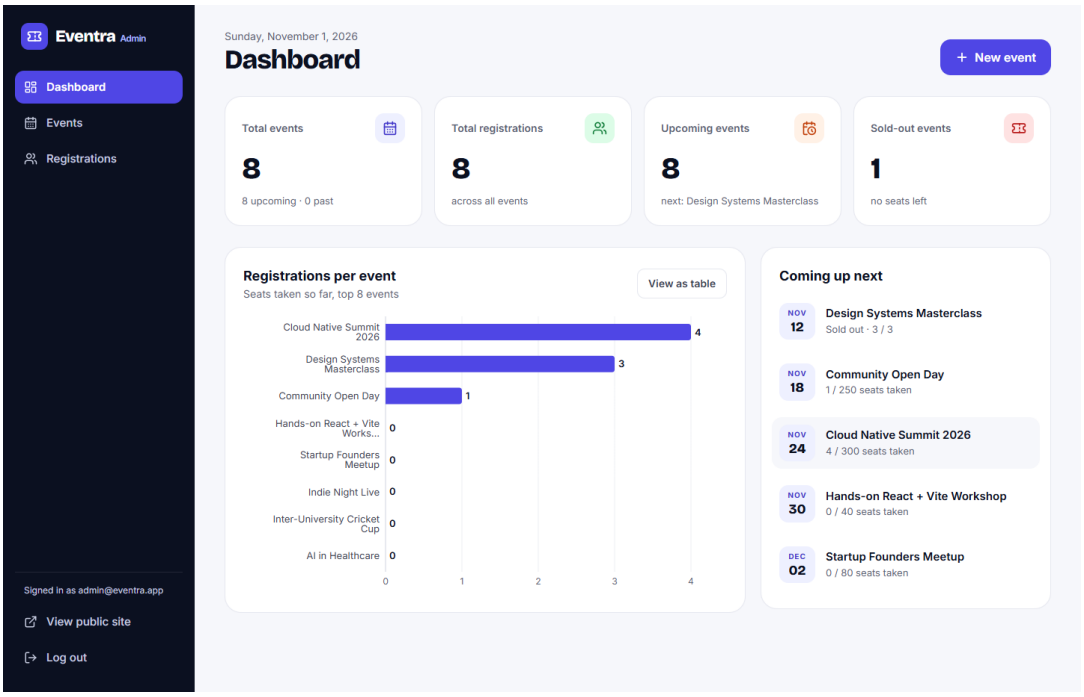


Figure 48: Admin dashboard with statistics and chart

Appendix B: Source Files

File	Purpose
frontend/Dockerfile, frontend/nginx/default.conf.template	Web tier image (multi-stage Node and Nginx) and reverse-proxy configuration
backend/Dockerfile	Application tier image (node:22-alpine, non-root)
docker-compose.yml	Local orchestration of the web, app and database services with health checks
.github/workflows/ci-cd.yml	CI/CD pipeline definition (test, build, push, deploy)
infra/terraform/*.tf	Infrastructure as code: network, security groups, ECS, ALB, RDS, ECR, IAM, secrets, monitoring
Repository	https://github.com/AunZaidii/Eventra-Event-Management-System